

GitLab CI/CD

DevOps Cheat Sheet · GCP Engineer Edition

PIPELINE STRUCTURE KEYWORDS

stages:	Ordered list of stage names	<i>e.g. [build, test, deploy]</i>
stage:	Assign a job to a stage	
script:	Shell commands to run in this job	<i>Required in every job</i>
before_script:	Runs before script in every job	<i>Setup / install dependencies</i>
after_script:	Runs after script even on failure	<i>Cleanup tasks</i>
image:	Docker image the job runs inside	<i>image: python:3.11</i>
services:	Extra containers for the job	<i>services: [postgres:15]</i>
tags:	Route job to a specific Runner	<i>Must match Runner registration</i>
needs:	Run as soon as listed jobs finish	<i>Skip waiting for full stage</i>
extends:	Inherit another job's config	<i>DRY — avoid repetition</i>
include:	Import external .yaml files	<i>Reuse shared CI config</i>

RULES, CONDITIONS & FLOW CONTROL

rules: - if:	Run job only when condition matches	<i>if: \$CI_COMMIT_BRANCH == "main"</i>
when: manual	Requires a human click to start	<i>Shows play button in UI</i>
when: on_failure	Only run if a previous job failed	<i>Good for notifications</i>
when: always	Always run — even after failures	<i>Cleanup / teardown jobs</i>
when: never	Disable the job entirely	
allow_failure: true	Pipeline stays green even if job fails	<i>Non-critical jobs</i>
retry: 2	Auto-retry job up to 2 times	
timeout: 10 minutes	Kill job if it exceeds time limit	
interruptible: true	Cancel if newer pipeline starts	<i>Saves runner minutes</i>
parallel: 5	Run 5 copies of the job simultaneously	
resource_group:	Prevent concurrent deploys to same env	

VARIABLES, ARTIFACTS & CACHE

variables:	Define custom key=value pairs	<i>Accessible as \$VAR_NAME</i>
\$CI_COMMIT_BRANCH	Current branch name	<i>Built-in variable</i>
\$CI_COMMIT_SHA	Full git commit hash	<i>Built-in variable</i>
\$CI_JOB_ID	Unique ID of this job run	<i>Built-in variable</i>
\$CI_PROJECT_NAME	Name of the GitLab project	<i>Built-in variable</i>
\$CI_REGISTRY_IMAGE	URL of the project's container registry	<i>Built-in variable</i>
\$CI_ENVIRONMENT_NAME	Name of the current environment	<i>Built-in variable</i>
artifacts: paths:	Files to save after job finishes	<i>Other jobs can download</i>
artifacts: expire_in:	How long to keep the files	<i>e.g. expire_in: 1 week</i>
artifacts: reports:	Special reports GitLab understands	<i>junit, coverage, sast, dast</i>
cache: paths:	Folders to persist between runs	<i>Speeds up installs</i>
cache: key:	Bucket name for the cache	<i>Same key = shared cache</i>

GITLAB + GCP DEPLOYMENT PATTERNS

docker build & push	Build image and push to Artifact Registry	<i>Use \$CI_REGISTRY_IMAGE</i>
--------------------------------	---	--------------------------------

gcloud auth activate-sa	Authenticate to GCP using SA key	<i>Store key as CI variable</i>
gcloud run deploy	Deploy to Cloud Run from pipeline	
gcloud app deploy	Deploy to App Engine	
gcloud container clusters...	Get GKE credentials for kubectl	
kubectl apply -f k8s/	Deploy manifests to GKE	<i>After gcloud get-credentials</i>
terraform apply	Run Terraform in a CI job	<i>Store state in GCS</i>
environment: name:	Link job to a GitLab environment	<i>Tracks deploy history</i>
environment: url:	URL shown in GitLab after deploy	
Workload Identity (OIDC)	Keyless GCP auth from GitLab CI	<i>Best practice — no SA keys</i>

GITLAB CI/CD + GCP EXAMPLE

```
stages: [build, test, deploy] variables: IMAGE: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA build: stage: build
image: docker:24 services: [docker:24-dind] script: - docker build -t $IMAGE . - docker push $IMAGE test: stage:
test image: python:3.11 script: [pip install -r requirements.txt, pytest] artifacts: reports: { junit:
report.xml } deploy-prod: stage: deploy image: google/cloud-sdk:slim script: - gcloud run deploy myapp --image
$IMAGE --region us-central1 environment: { name: production, url: https://myapp.run.app } rules: - if:
$CI_COMMIT_BRANCH == "main" when: manual
```

Tip: Use Workload Identity Federation (OIDC) for keyless GCP auth — no JSON keys needed.

docs.gitlab.com/ee/ci